
EMPLOYMENT MANAGEMENT SYSTEM

The Frontend

Routing, stores, TanStack Query, and every page

REACT ARCHITECTURE • 2 OCTOBER 2026

Contents

01	1. Stack and versions	3
02	2. Bootstrap: <code>index.html</code> -> <code>main.jsx</code> -> <code>App.jsx</code>	3
03	3. Routing (<code>App.jsx</code>)	4
04	4. <code>RequireAuth</code> - the three states	5
05	5. <code>authStore</code> - the session	6
06	6. <code>uiStore</code> - everything cosmetic	6
07	7. <code>usePermissions</code> - roles to a boolean	7
08	8. <code>api/http.js</code> - the single network boundary	8
09	9. TanStack Query - how server state is handled	9
10	10. <code>Layout</code> - the shell	10
11	11. Pages in detail	11
12	12. Modals	12
13	13. Tailwind conventions	13
14	14. Dev proxy and deployment	13
15	15. Linting and building	14

How the React app is wired: the entry point, routing, the two Zustand stores, the permissions hook, TanStack Query, and every page.

1. Stack and versions

Package	Version	Why it is here
<code>react</code> / <code>react-dom</code>	19.2	UI
<code>react-router-dom</code>	7.18	routing, nested layouts, <code>Outlet</code> , <code>Navigate</code>
<code>@tanstack/react-query</code>	5.103	server state: caching, loading flags, refetch, invalidation
<code>zustand</code>	5.0	client state: token, user, theme, sidebar, toasts
<code>vite</code>	8.3	dev server + build
<code>@tailwindcss/vite</code>	4.3	Tailwind v4 as a Vite plugin (no <code>tailwind.config.js</code>)
<code>eslint</code>	1.81	linter

There is **no** component library, no CSS framework beyond Tailwind, and no form library - forms are plain `useState` plus HTML validation. Modals are hand-rolled `position: fixed` overlays, not a UI kit.

2. Bootstrap: `index.html` -> `main.jsx` -> `App.jsx`

`index.html` is nearly empty. It contains `<div id="root"></div>` and one module script pointing at `/src/main.jsx`.

```
// main.jsx
const queryClient = new QueryClient();

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </StrictMode>,
);
```

- `createRoot` is React 18+ API.
- `StrictMode` double-invokes render and effects **in development only**, to surface side effects in effects. It has no effect on the production build.

- `QueryClientProvider` must wrap everything because `useQuery`, `useMutation` and `useQueryClient` all read the client from React context.

Dark mode before React boots

`index.css` (Tailwind v4) enables dark mode with a class strategy. `uiStore` reads `localStorage.theme` at module-initialisation time (`getInitialTheme()`), so the very first paint already has the right theme.

3. Routing (App.jsx)

```
const router = createBrowserRouter([
  {
    path: "",
    element: <RequireAuth />,
    children: [
      {
        path: "",
        element: <Layout />,
        children: [
          { index: true, element: <DashboardPage /> },
          { path: "employees", element: <EmployeesPage /> },
          { path: "departments", element: <DepartmentPage /> },
          { path: "attendance", element: <AttendancePage /> },
        ],
      },
    ],
  },
  { path: "/login", element: <LoginPage /> },
  { path: "/register", element: <RegisterPage /> },
]);
return <RouterProvider router={router} />;
```

Read it as a tree of nesting:

```
/           → RequireAuth → Layout → DashboardPage
/employees  → RequireAuth → Layout → EmployeesPage
/departments → RequireAuth → Layout → DepartmentPage
/attendance → RequireAuth → Layout → AttendancePage
/login      → LoginPage   (no auth, no layout)
/register   → RegisterPage (no auth, no layout)
```

- `RequireAuth` and `Layout` are **layout routes**. They render once and their `<Outlet />` swaps the child page, so the sidebar and header do not remount when you navigate. That is the entire point of the nesting.
- `index: true` means "the empty path", i.e. `/`.

- `createBrowserRouter` uses the **HTML5 History API**, so URLs are real paths. That is why deploying to a static host needs the `vercel.json` rewrite.

The exact gate sequence for an employee

URL visited by an <code>EMPLOYEE</code>	What happens
<code>/login</code>	<code>if (token) return <Navigate to="/" /></code> -> they land on <code>/</code>
<code>/</code>	Dashboard renders, then <code>if (!canManage) return <Navigate to="/attendance" replace /></code>
<code>/employees</code>	same redirect from <code>EmployeesPage</code>
<code>/departments</code>	same redirect from <code>DepartmentsPage</code>
<code>/attendance</code>	allowed

`replace` on the `<Navigate>` matters: it **replaces** the history entry instead of pushing one, so the browser Back button does not bounce them into a loop.

All three manager-only pages use the *same* pattern, and all three put the redirect **after** the hooks (queries and mutations are declared first, `if (!canManage) return <Navigate ... />` comes after). That ordering was fixed deliberately - returning early before the hooks would violate the rules of hooks and change hook order between renders.

4. RequireAuth - the three states

Covered in detail in `02-authentication.md` section 9. The short version:

Condition	Output
no token	<code><Navigate to="/login" replace /></code>
token, no user yet	<code>Loading ...</code>
token + user	<code><Outlet /></code>

```
const token = useAuthStore((s) => s.token);
const user = useAuthStore((s) => s.user);
const loadUser = useAuthStore((s) => s.loadUser);

useEffect(() => { if (token && !user) loadUser(); }, [token, user, loadUser]);
```

JSX

Three separate selectors, not `const { token, user } = useAuthStore()`. Each selector subscribes the component to **one** field, so an update to `theme` or `toasts` does not re-render `RequireAuth`. This is Zustand's headline feature and the codebase uses it consistently.

5. authStore - the session

```
export const useAuthStore = create((set, get) => ({
  token: localStorage.getItem("ems_token") || null,
  user: null,

  setSession: (token) => { localStorage.setItem("ems_token", token); set({ token }); },
  logout: () => { localStorage.removeItem("ems_token"); set({ token: null, user: null }); },
  loadUser: async () => { /* GET /api/auth/me */ },
}));
```

`create` is Zustand's store factory. `set` replaces state, `get` reads it without subscribing - which is why `api()` uses `useAuthStore.getState().token` instead of the hook form.

State shape:

Field	Persisted?	Source
<code>token</code>	yes, <code>localStorage["ems_token"]</code>	login/register response
<code>user</code>	no - always re-fetched	<code>GET /api/auth/me</code>

The split is deliberate: the token is durable, but identity and roles are always re-confirmed with the server so a revoked account or changed role takes effect immediately.

6. uiStore - everything cosmetic

```
export const useUIStore = create((set) => ({
  isSidebarOpen: true,
  toggleSidebar: () => set((s) => ({ isSidebarOpen: !s.isSidebarOpen })),
  toasts: [],
  removeToast: (id) => set((s) => ({ toasts: s.toasts.filter(t => t.id !== id) })),
  pushToast: (message, type = "success") => { ... },
  theme: getInitialTheme(),
  toggleTheme: () => { ... },
}));
```

- `set((state) => ...)` with a **function** is the updater form: it receives the current state, which is what you need when deriving new state from old state (toggling a boolean, filtering a list).

- `pushToast` assigns `id = Date.now()` and schedules a `setTimeout(4000)` that removes it. Two toasts created in the same millisecond would share an id - a known theoretical collision.
- `toggleTheme` does three things at once: flips the value, **persists** it, and toggles the `dark` class on `<html>` directly. The DOM update inside the store is unusual (a side effect in a state setter) and is duplicated by the `useEffect` in `Layout` - belt and braces, but it means the class is applied even before `Layout` mounts.

`Toasts.jsx` renders the array in a fixed bottom-right container, green for `success` and red for `error`, and clicking a toast removes it immediately.

The dark-mode class

`Layout.jsx:25`:

```
useEffect(() => {
  document.documentElement.classList.toggle("dark", theme === "dark");
}, [theme]);
```

Tailwind v4 with the class strategy darkens every element that has a `dark:` variant. `theme` is read with `const { isSidebarOpen, toggleSidebar, theme, toggleTheme } = useUIStore();` - destructuring the **whole** store, which does subscribe the component to every field (acceptable here, but inconsistent with the selector style used elsewhere).

7. usePermissions - roles to a boolean

```
export function usePermissions() {
  const roles = useAuthStore((s) => s.user?.roles) ?? [];
  return {
    roles,
    canManage: roles.includes("ROLE_ADMIN") || roles.includes("ROLE_MANAGER"),
  };
}
```

- `?.` guards against `user` being null during the `Loading ...` phase; `?? []` then guarantees `roles.includes` is safe.
- The strings include the `ROLE_` prefix because that is exactly what `User.getAuthorities()` puts into `SimpleGrantedAuthority`.
- This hook is a **convenience, not a security boundary**. It hides UI. The backend still enforces everything - if you deleted `usePermissions` tomorrow, the app would still be safe, just noisier.

Early on this hook returned a bare boolean; it now returns an object. Any caller written for the old shape (`const canManage = usePermissions()`) would break, which is why every call site now destructures.

8. api/http.js - the single network boundary

Every request in the app goes through the private `api()` helper. It:

1. reads the token from the store (non-hook `getState()`),
2. sets `Content-Type` **only if there is a body**,
3. sets `Authorization` **only if there is a token**,
4. on `401` with a token present: logs out and hard-redirects to `/login`,
5. on any other non-2xx: throws `new Error(message)`,
6. returns `null` for `204`, otherwise `response.json()`.

Then thin named wrappers give every endpoint a function. There is no axios, no interceptors, no base URL constant - paths are literal strings starting with `/api`, which is what makes the Vite proxy work with zero configuration.

Grouped by feature

```
// Employees      : fetchEmployees, fetchEmployee, searchEmployee, createEmployee,
//                  updateEmployee, deleteEmployee, fetchAllEmployees
// Departments    : fetchDepartments, fetchDepartmentsPage, fetchDepartmentEmployees,
//                  createDepartment, updateDepartment, deleteDepartment
// Attendance     : searchAttendance, fetchMyAttendance, checkIn, checkOut
// Auth           : loginUser, registerUser
```

Two naming inconsistencies worth being able to explain:

- `fetchDepartments` is the **unpaged** `size=100` version, while `fetchDepartmentsPage` is the paginated one. The names read backwards.
- Employee list is `fetchEmployees` (paged) while the all-rows version is `fetchAllEmployees`.

fetchMyAttendance builds a query string properly

```
export async function fetchMyAttendance(page, size = 10, startDate, endDate) {
  const params = new URLSearchParams({ page, size });
  if (startDate) params.append("startDate", startDate);
  if (endDate)   params.append("endDate", endDate);
  return api(`/api/attendance/me?${params}`);
}
```

`URLSearchParams` encodes the values and skips the empty ones, which is why the default `startDate` / `endDate` of `undefined` never appear in the URL. Compare `searchEmployee(name)`, which interpolates the name raw - the better pattern is used here and the worse one there.

9. TanStack Query - how server state is handled

The three hooks

Hook	Purpose	Returns
<code>useQuery</code>	read data	<code>{ data, isLoading, isFetching, isError, error }</code>
<code>useMutation</code>	write data	<code>{ mutate, isPending, isError }</code>
<code>useQueryClient</code>	reach the cache	<code>{ invalidateQueries, clear }</code>

Query keys

The key is the cache identity. Two calls with the same key share one network request and one cached value. The keys in this app:

Key	Used by
<code>["employees", page, search]</code>	<code>EmployeesPage</code>
<code>["departments", page]</code>	<code>DepartmentsPage</code>
<code>["departments"]</code>	<code>EmployeeFormModal</code> dropdown, attendance filter dropdown
<code>["department-employees", id]</code>	<code>DepartmentDetailsModal</code>
<code>["attendance", "search" "me", applied, page]</code>	<code>AttendancePage</code> main table
<code>["attendance-today", "search" "me", id]</code>	<code>AttendancePage</code> clock card
<code>["employee-list"]</code>	<code>AttendancePage</code> manager dropdown
<code>["departments-list"]</code>	<code>AttendancePage</code> department filter
<code>["dashboard-employees"]</code> , <code>["dashboard-departments"]</code> , <code>["dashboard-attendance"]</code>	<code>DashboardPage</code>

Because `invalidateQueries({ queryKey: ["employees"] })` treats the key as a **prefix**, invalidating `["employees"]` clears every page and every search state at once. Invalidation marks entries stale and refetches what is currently mounted.

isLoading vs isFetching

- `isLoading` = **no data yet** (first load).
- `isFetching` = a request is in flight, including a refetch with data already shown.

`AttendancePage` uses both:

```
placeholderData: (prev) => prev,
...
const isApplying = isFetching && !isLoading;
```

JSX

`placeholderData: (prev) => prev` keeps the previous page's rows on screen while the next page loads, so paginating does not blank the table.

enabled

`enabled: canManage` (and `enabled: canManage ? !!effectiveClockEmployeeId : true`) means the query does not fire at all until the condition holds. This is how the app avoids guaranteed-403 requests from employees.

Mutations are where cache invalidation happens

Every mutation follows the same shape:

```
onSuccess: () => {
  pushToast("Employee Deleted");
  queryClient.invalidateQueries({ queryKey: ["employees"] });
},
onError: (error) => pushToast(error.message, "error"),
```

JSX

There is **no optimistic update** anywhere: the UI waits for the server, then refetches. Simpler and always correct, at the cost of a round trip.

10. Layout - the shell

Structure: fixed sidebar (`w-56` open / `w-14` collapsed) + flex column with a header and a `<main>` containing `<Outlet />`.

```
const links = [
  { to: "", label: "Dashboard", canManage: true },
  { to: "employees", label: "Employees", canManage: true },
  { to: "departments", label: "Departments", canManage: true },
  { to: "attendance", label: "Attendance" },
];
...
{links.filter((link) => !link.canManage || canManage).map((link) => (...))}
```

JSX

`!link.canManage || canManage` means: links with **no** `canManage` flag are always shown; links with the flag are shown only to managers. Attendance has no flag, so everyone sees it.

- `NavLink` with `end={link.to === ""}` - `end` makes `/` match only the exact path. Without it, `/` would stay highlighted on every page because it is a prefix of all of them.

- `className={({ isActive }) => ...}` is the function form; `NavLink` passes the match state.
- The collapsed sidebar shows only `link.label[0]` - the first character as a placeholder icon.
- `Toasts` is rendered once in `Layout`, so toasts survive navigation.

The header shows `user.username`, `user.roles[0].replace("ROLE_", "")`, a theme toggle with two inline SVGs, and the Logout button.

11. Pages in detail

LoginPage / RegisterPage

Both are self-contained: local `useState` for each field, a `submitting` flag that disables the button, `pushToast(error.message, "error")` on failure, and

```
if (token) return <Navigate to="/" replace />;
```

JSX

so a logged-in visitor is bounced. Both call `setSession(token)` then `await loadUser()` then `navigate("/")`. The `await loadUser()` is not optional - without it the app would navigate to `/` with a token but no user, and `RequireAuth` would sit on `Loading ...` until some other trigger refetched.

`RegisterPage` also displays the hint "Email must match an existing employee" - surfacing the backend's rule in the UI so the `404` is not a surprise.

DashboardPage

Three queries, all `enabled: canManage`:

```
queryKey: ["dashboard-employees"], queryFn: fetchAllEmployees
queryKey: ["dashboard-departments"], queryFn: fetchDepartments
queryKey: ["dashboard-attendance"], queryFn: () => searchAttendance({}, 0, 1000)
```

JSX

- `fetchAllEmployees` -> `GET /api/employees` (all rows, unpaged).
- `searchAttendance({}, 0, 1000)` -> `POST /api/attendance/search` with an **empty filter object and size 1000**. This is the "no filters" case: `filters` has no `null` keys, so no `Specification` is added and every row is returned.
- `totalAttendanceCounts` comes from `attendance?.totalElements ?? 0` - the `count(*)` from the `Page`, not `content.length`.
- Status counts are computed **client-side** by looping the fetched rows and incrementing `totalStatusCounts[att.status]`, guarded by `if (totalStatusCounts[att.status] !== undefined)` so an unknown status from the server cannot create a `NaN` key.

- `recent8` sorts a copy (`[ ... attendanceRow ]`) **descending by date string** and takes 8. String comparison works because ISO dates sort lexicographically.
- `filterByDept` counts employees per department client-side: `employeeList.filter(e => e.department?.id === dep.id).length`.
- `statusStyles` is a plain map of status -> Tailwind classes; `statusLabel` replaces `_` with a space and lowercases (`HALF_DAY` -> `"half day"`).
- `formatTime(value) => value.slice(11, 16)` - ISO string slicing again.

This page is entirely client-side aggregation over three already-fetched responses - there is no statistics endpoint.

EmployeesPage

Already covered in `03-employees.md`. The structural points: one `useQuery` whose `queryFn` switches between paged and search endpoints, a `useMutation` for delete, `page` state, two search states (`searchInput` vs `search`), a `modal` state that is either `null`, `{mode: "create"}` or `{mode: "edit", employee}`, and `{!isSearching && (...)}` hiding the pager while searching.

DepartmentsPage

`{mode: "create" | "update", dep: null}` state shape, plus a separate `details` state that opens the read-only details modal. The "View" button is **always** rendered (not wrapped in `canManage`), while Edit/Delete are - legacy leftovers from when the page was visible to employees. It has no effect now because the whole page is manager-only.

AttendancePage

Covered in `05-attendance.md`. The distinctive parts are the derived `effectiveClockEmployeeId`, the dual-endpoint query, and the three invalidations in `invalidateAttendance()`.

12. Modals

Three hand-rolled modals, all the same pattern:

```
<div className="fixed inset-0 z-40 flex items-center justify-center bg-black/40">
  <form className="w-full max-w-md rounded-lg bg-white p-6 shadow-xl dark:bg-slate-800">
```

JSX

The parent page owns visibility with a `modal` state variable and renders `{modal && <EmployeeFormModal ... onClose={() => setModal(null)} />}`. The modal itself holds no "open" flag - being mounted is being open. Unmounting is how it closes, which also discards the form state.

Notable details:

- `fixed inset-0` covers the viewport; `bg-black/40` is the dimmed backdrop. It is **not click-to-close** - only the Cancel button or an action closes it.
- `z-40` for modals, `z-50` for toasts, so a toast appears above a modal.
- `const inputClass = " ... "` is defined once per modal and reused on every input, which is the local styling convention.
- `DepartmentDetailsModal` is not a `<form>` - it is a `<div>` containing a read-only table, plus one Close button.

13. Tailwind conventions

- Colours use the `slate` neutral ramp plus semantic accents: `sky` (primary action), `emerald` / `green` (success), `violet` (dashboard accent), `red` (destructive), `amber` (warning).
- Every themed element has a `dark:` twin, e.g. `bg-white dark:bg-slate-800`.
- Spacing is `p-4` / `p-6` / `p-8`, rounded corners `rounded` / `rounded-lg`, and transitions `transition-all`.
- Focus rings are `focus:ring-2 focus:ring-sky-500` (no `focus:outline-none` conflict).

14. Dev proxy and deployment

`vite.config.js`:

```
server: { proxy: { "/api": "http://localhost:8080" } }
```

JS

In development the browser calls `/api/ ...` on the **Vite dev server** (port 5173), and Vite forwards those requests to Spring Boot on 8080. That is why the frontend never hardcodes `http://localhost:8080` and why there is no CORS configuration on the backend - same-origin in dev, so the browser never sees a cross-origin request.

In production the built files are static, so `vercel.json` provides the equivalent rewrite:

```
{ "rewrites": [ { "source": "/api/:path*",
                  "destination": "https://YOUR-EMS-BACKEND.onrender.com/api/:path*" } ] }
```

JSON

`YOUR-EMS-BACKEND.onrender.com` is a **placeholder** - it must be replaced with the real backend URL. Also note that `vercel.json` only rewrites `/api`; a static host must separately serve `index.html` for client-side routes like `/employees` on a hard refresh, otherwise React Router has nothing to render.

15. Linting and building

```
npx oxlint src      # 0 warnings, 0 errors
npm run build       # vite build → dist/
npm run dev         # dev server with the /api proxy
```

BASH

Both were run clean after the role-gating changes.